

Animal Cost, Lifecycle & Health Intelligence API

Everything added this build: a Finance ledger under the animal & feed & health modules, batch-tracked Pharmacy and Feed stores, a species lifecycle and breeding-eligibility engine, a health & growth alert system, and sale-readiness/profitability scoring — plus the reports and audit trail that tie it together.

56

NEW ENDPOINTS

9

APPS TOUCHED

14

REPORT VIEWS

233

TOTAL API PATHS

Contents

1. Conventions & auth
2. Finance ledger
3. Animal acquisition & growth
4. Pharmacy & batches
5. Feed batches & allocation
6. Species lifecycle
7. Breeding eligibility
8. Weight reference ranges
9. Health & performance alerts
10. Sale readiness & profitability
11. Reports
12. Audit & overrides

Conventions & auth

Applies uniformly across every endpoint below — not repeated per entry.

Authentication

JWT in an httpOnly cookie
(`client_access_token`), set at login. No bearer header — the cookie rides with every request.

Response envelope

`{ success, message, data }`. Paginated list endpoints add `num_pages`, `current_page`, `total_items`, `has_next`, `has_previous`.

Farm scoping

Nearly every path takes a `farm_id`. The server always re-checks that farm belongs to the caller's organization — a `farm_id` from another org 404s, never 200s with someone else's data.

Permission model

Org owners pass automatically. Everyone else needs the named permission attached to a `Role` via `UserRole`. Endpoints marked `permission` below name theirs.

Master data: system vs. farm

Categories, drugs, feed types, and rule tables ship with sane system defaults

(`is_system=true`) and can be overridden per farm — the more specific record always wins.

Seed endpoints

One-time, superuser-only, idempotent. Run once per environment to populate categories/units/default rules; safe to call again — nothing duplicates.

01 • Finance ledger

Every cost this build introduces — acquisition, feed, treatment, breeding — lands here as a `Transaction`. Nothing else in the system stores "the total cost"; totals are always recomputed by summing transactions, so a historical entry never silently moves when a price changes later.

METHOD	PATH	PURPOSE
POST	/api/finance/transaction-category/seed/	Seed 16 default categories (expense, 8 income)
GET	/api/finance/transaction-category/	List categories, optional <code>?type=</code> filter
POST	/api/finance/transaction/	Record a manual transaction (labour, ad-hoc costs)
GET	/api/finance/transaction/{page}/{page_size}/{farm_id}	List transactions, filter by <code>animal_id / group_id / type</code>
GET	/api/finance/animal-financial-summary/{animal_id}/	Cost breakdown, income, estimated profit/loss for one animal
GET	/api/finance/animal-cost-timeline/{animal_id}/	Chronological ledger for one animal

POST /api/finance/transaction/

Manual ledger entry — for costs that don't originate from another module's automatic posting (labour, one-off purchases).

Permission: `add_finance` (bypassed for org owners)

REQUEST BODY

farm_id	int	required
type	"expense" "income"	required
category_id	int	required
amount	float	required
transaction_date	date	required
animal_id / group_id	int	optional
payment_status	enum	default "paid"
notes	string	optional

RESPONSE · DATA

```
{
  "id": 41,
  "type": "expense",
  "amount": 6500,
  "category": "Feed"
}
```

GET `/api/finance/animal-financial-summary/{animal_id}/`

The number every other "cost per animal" report reuses. Acquisition/opening value is folded in exactly once, whether or not it was also posted as its own transaction.

RESPONSE · DATA

```
{
  "animal_id": 128,
  "tag_id": "GT-0042",
  "acquisition_or_opening_value": 475000,
  "cost_breakdown": { "Acquisition": 475000, "Feed": 6500, "Treatment": 4200 },
  "total_cost_to_date": 485700,
  "income_generated": 12000,
  "estimated_current_value": 520000,
  "estimated_profit_or_loss": 46300
}
```

Behind the scenes

- Acquisition value counts once — if an `Acquisition` -category transaction already exists, the raw `acquisition_cost` field is not added a second time.
- $\text{estimated_profit_or_loss} = \text{current_estimated_value} + \text{income_generated} - \text{total_cost_to_date}$

02 · Animal acquisition & growth

How an animal entered the farm, turned into a costed baseline — and, separately, its weight/growth trend once it's on the ground.

CROSSBREED FIELDS (EXISTING CREATE ENDPOINTS)

`/api/animals/animal/` and `/animal/v2/` gained three fields: `breed_type` ("purebred" | "crossbred"), `sire_breed_id`, `dam_breed_id`. Crossbred requires both breed IDs; purebred forbids them — enforced server-side, not just a UI hint.

POST `/api/animals/animal/{animal_id}/acquisition/`

One endpoint, four entry methods. The animal's own `source_type` decides which cost formula runs and which Finance category the result lands in. Safe to call again later (e.g. to attach a receipt) — updates the same record, never double-posts the transaction.

Permission: `update_animal_details`

FORMULA BY SOURCE_TYPE

SOURCE_TYPE	FORMULA	POSTED AS
purchased	price + transport + vet-inspection + other	Acquisition (expense)
imported	price + shipping + customs + quarantine + vet-cert + insurance + other	Acquisition (expense)
born	dam-feeding + pregnancy-treatment + delivery + breeding cost, <i>each kept separate</i>	Feed / Veterinary Service / Breeding — never "Acquisition" (it wasn't a purchase)
opening_record	estimated_opening_value as given	<code>Animal.opening_value</code> only — no transaction (nothing was spent through this system)

EXAMPLE — PURCHASED

```
POST /api/animals/animal/42/acquisition/
{
  "supplier": "ACME Livestock",
  "purchase_price": 450000,
  "transportation_cost": 20000,
  "veterinary_inspection_cost": 5000,
  "purchase_date": "2026-07-12"
}
→ acquisition_cost: 475000, one "Acquisition" transaction posted
```

GET `/api/animals/animal-growth/{animal_id}/`

Weight gain, average daily gain, % change, and cost-per-kg-gained — the last one pulled live from the Finance ledger, not a separate stored figure.

RESPONSE · DATA

```
{
  "animal_id": 42,
  "weight_gain_kg": 3,
  "average_daily_gain_kg": 0.1,
  "percentage_weight_change": 30.0,
  "cost_per_kg_gained": 100.0,
  "weight_trend": [
    { "date": "2026-06-25", "weight": 10 },
    { "date": "2026-07-25", "weight": 13 }
  ]
}
```

03 • Pharmacy & batches

A drug store that didn't exist before this build. Drugs are master data (system + farm-defined); every purchase is its own `DrugBatch` with its own price, expiry, and remaining quantity — batches are never averaged together, so a price change today can't rewrite the cost of a dose given last month.

METHOD	PATH	PURPOSE
POST	/api/pharmacy/drug-master/seed/	Seed 8 categories + 6 common drugs
GET	/api/pharmacy/drug-category/	List drug categories
GET	/api/pharmacy/drug/{page}/{page_size}/{farm_id}	List drugs visible to a farm (system + own)
POST	/api/pharmacy/drug/	Register a farm-specific drug
POST	/api/pharmacy/drug-batch/	Receive a new batch into stock
GET	/api/pharmacy/drug-batch/{page}/{page_size}/{farm_id}	List batches, filter by <code>drug_id / status</code>
POST	/api/pharmacy/pharmacy-alert/scan/{farm_id}/	Re-check every active batch for expiry/stock alerts
GET	/api/pharmacy/pharmacy-alert/{page}/{page_size}/{farm_id}	List batch alerts
GET	/api/pharmacy/animal-withdrawal/{farm_id}/	Animals currently inside a drug withdrawal window

POST /api/pharmacy/drug-batch/

Receiving stock. Cost-per-base-unit is computed once, at intake, from this batch's own price.

REQUEST BODY

farm_id, drug_id	int	required
batch_number	string	required
quantity_received	float	required
purchase_price	float	required
expiry_date	date	required
supplier, storage_location	string	optional
minimum_stock_level	float	optional — drives low-stock alert

RESPONSE • DATA

```
{
  "id": 7,
  "drug": "Penicillin",
  "batch_number": "PEN-001",
  "quantity_available": 1000,
  "cost_per_base_unit": 50.0
}
```

POST `/api/pharmacy/pharmacy-alert/scan/{farm_id}/`

Sweeps every non-depleted batch for the farm and opens/closes alerts as conditions change.

Alert types

- `drug_expired` — **high priority** — expiry date has passed
- `drug_expiring_soon` — monitor — expires within 30 days
- `drug_low_stock` — monitor — at or below `minimum_stock_level`
- `drug_out_of_stock` — attention required

Each alert auto-resolves the next time this scan runs and the condition no longer holds — nothing needs manual closing once it's fixed.

04 • Feed batches & allocation

Feed follows the same batch discipline as drugs, plus one problem drugs don't have: feed is bought in bags, tonnes, bales — never the unit it's costed or fed in. And when a whole group eats from one bag, that cost has to land on individual animals somehow.

POST /api/feed/feed-batch/

Buy by the bag, cost by the kilogram — converted once at intake.

REQUEST BODY

feed_type_id, base_unit_id	int	required
purchase_unit	bag kg tonne bale sack litre container other	required
package_size	float — e.g. 50 (kg per bag)	required
number_of_packages	float — e.g. 20 bags	required
purchase_price	float — total, all packages	required
expiry_date	date	optional

RESPONSE · DATA

```
{
  "id": 3,
  "feed_type":
  "Dairy
  Concentrate",

  "total_quantity
  _base_unit":
  1000,

  "cost_per_base_
  unit": 360.0,

  "cost_per_packa
  ge": 18000.0
}
```

20 BAGS × 50KG @
£18,000/BAG → £360/KG
— MATCHES THE SPEC'S
OWN WORKED EXAMPLE
EXACTLY.

Side effect — the farm's existing `FeedInventory` total for this feed type increments automatically. Every report and screen built on `FeedInventory` before this batch system existed keeps working unchanged.

POST `/api/feed/feed-issue/` extended

Existing issuance endpoint, now batch- and cost-aware — entirely opt-in. Omit the new fields and it behaves exactly as before (no cost computed, no allocation required).

<code>feed_batch_id</code>	<code>int</code>	optional — enable issuance
<code>feeding_period</code>	<code>morning afternoon evening full_day</code>	optional
<code>fed_by_id</code>	<code>int (user)</code>	optional — distributed by <code>issued_by</code>
<code>allocation_method</code>	<code>equal weight_based consumption_based life_stage_based manual</code>	required only when <code>target_type</code> batch is given
<code>manual_allocations</code>	<code>[{ animal_id, quantity }]</code>	required by <code>manual / consumption_based</code>

Group cost splitting — never defaults to equal

- `equal` — even split across active group members
- `weight_based` — proportional to each animal's latest recorded weight
- `life_stage_based` — proportional to a per-stage weighting table (Grower 1.0×, Mature 1.2×, Newborn 0.3×, ...)
- `manual / consumption_based` — exactly the quantities you supply; omitting `manual_allocations` is a 400, not a silent equal-split

Every group issuance writes one `FeedCostAllocation` row per animal (the audit trail of which method produced which number) and posts one Finance transaction per animal — all inside a single database transaction, so a bad allocation can't leave stock deducted with no matching cost record.

05 • Species lifecycle

Newborn → Suckling → Weaned → Juvenile → Grower → Mature → Breeding Eligible → Senior, plus Pregnant/Lactating as condition-driven overrides — configured per species, auto-suggested from age/weight/sex, and only ever changed with a recorded reason.

METHOD	PATH	PURPOSE
POST	/api/admin/lifecycle/seed/	Seed stage ladders for all 7 species
GET	/api/admin/lifecycle/stages/{species_id}/	List configured stages for a species
GET	/api/admin/lifecycle/animal/{animal_id}/	Current stage + suggested stage + change history
POST	/api/admin/lifecycle/animal/{animal_id}/refresh/	Recompute from current age/weight/status
POST	/api/admin/lifecycle/animal/{animal_id}/override/	Force a specific stage, with a required reason

POST

/api/admin/lifecycle/animal/{animal_id}/override/

The only lifecycle write that doesn't come from the automatic engine.

Permission: `update_animal_details` · Query params: `stage` , `reason` (both required)

Sold/Deceased are read straight from `Animal.status` and can't be overridden into something else — a sold animal is always "Sold" regardless of age or weight.

06 · Breeding eligibility

Replaces a check that used to be just "is this animal female?" with age, weight, postpartum interval, lifetime-birth caps, health status, and lifecycle stage — resolved per species, with farm- and breed-level overrides.

METHOD	PATH	PURPOSE
POST	/api/reproductions/breeding-rule/seed/	Seed default rules for all 7 species
GET	/api/reproductions/breeding-rule/{species_id}/	View system + farm rule
POST	/api/reproductions/breeding-rule/	Set a farm- or breed-specific override
GET	/api/reproductions/breeding-eligibility/{animal_id}/	Dry-run check — is this animal eligible, and why not

GET /api/reproductions/breeding-eligibility/{animal_id}/

Called before showing a "record insemination" screen, so the UI can explain a block before the user hits submit.

RESPONSE · DATA

```
{
  "animal_id": 91,
  "is_eligible": false,
  "reasons": [
    "Animal has not reached the minimum breeding age (7 months).",
  ],
  "rule_source": "system"
}
```

Checked, in order

- Sex, alive/active status, not already pregnant
- Min/max breeding age & min breeding weight (species/breed/farm rule)
- Postpartum recovery interval since last birth
- Lifetime birth cap, if configured
- Quarantine, sick health status, restricted flag, lifecycle stage (Retired/Sold/Deceased)
- Pregnant + lactating together — *allowed by default*, blocked only if the farm's own policy says so

07 • Weight reference ranges

Age-banded expected weight per species (farm/breed-overridable) — the baseline the health alert engine checks every recorded weight against.

METHOD	PATH	PURPOSE
POST	/api/admin/weight-range/seed/	Seed age-band ranges for all 7 species
GET	/api/admin/weight-range/{species_id}/	List configured ranges
GET	/api/admin/weight-range/animal/{animal_id}/	The specific band that applies to one animal right now

08 • Health & performance alerts

Decision support, not diagnosis — every alert names its evidence and a recommended review, never a verdict.

METHOD	PATH	PURPOSE
POST	/api/health/treatment/ extended	Now accepts drug/batch/dose/route — deducts stock and posts cost when a batch is given
POST	/api/health/treatment/external/	Emergency medication given before it entered inventory
POST	/api/health/treatment/{id}/reconcile-to-inventory/	Retroactively log an external dose as a (fully-consumed) batch
POST	/api/health/health-alert/scan/{animal_id}/	Run all 6 detectors for one animal
GET	/api/health/health-alert/{page}/{page_size}/{farm_id}	List alerts, filter by status / severity
POST	/api/health/health-alert/{id}/resolve/	Close an alert with resolution notes

POST

/api/health/treatment/ extended

A treatment is still valid as a plain clinical note. Add drug_id + drug_batch_id + quantity_administered and it becomes a costed, stock-deducting event too.

drug_id, drug_batch_id, quantity_administered	all optional, but batch requires quantity	
dose, frequency, duration_days, administration_route	optional	
administered_by_id / prescribed_by_id	int (user)	optional — separate from created_by

When a batch is given

- Deducts quantity_administered from that specific batch (insufficient stock → 400, nothing partially applied)
- treatment_cost = quantity_administered × batch.cost_per_base_unit , posted to Finance under "Treatment"
- withdrawal_end_date auto-computed from the drug's configured withdrawal period — this is what later blocks a premature sale

POST /api/health/treatment/external/

For medication given before it was formally received into stock. Deliberately bypasses batch deduction — so it's gated behind its own permission, separate from ordinary treatment recording.

Permission: `external_medication_override` — distinct from normal treatment-create rights, specifically to stop routine use from sidestepping inventory tracking.

<code>external_drug_name, quantity_administered, unit</code>	required
--	-----------------

<code>external_unit_cost, external_source, external_reason</code>	required
---	-----------------

Cost is posted the same as any other treatment. If the drug is later registered properly, `reconcile-to-inventory` creates a matching batch — pre-filled to zero remaining, since it was already used — purely for audit completeness.

THE SIX DETECTORS

ALERT_TYPE	TRIGGER	SEVERITY
<code>low_weight_for_age / high_weight_for_age</code>	Outside the configured weight-range band	attention required / monitor
<code>sudden_weight_loss</code>	≥8% drop since the previous measurement	high priority
<code>no_weight_recorded</code>	Nothing logged in 60 days	monitor
<code>poor_feed_conversion</code>	Feed cost incurred with ≈zero weight gain over 30 days	attention required
<code>reproductive_concern</code>	Past recommended breeding age, no reproductive event on record	monitor
<code>production_decline</code>	7-day milk average <85% of the prior 30-day average	attention required

Every alert auto-resolves the next time its scan runs and the condition clears — fixing the underlying weight, stock, or output closes it without a manual step.

09 • Sale readiness & profitability

Six-state readiness, not a boolean — and pregnancy is never a hardcoded block, per farm policy.

METHOD	PATH	PURPOSE
POST	/api/movement-records/sale-policy/seed/	Seed default target weights/ages for all 7 species
POST	/api/movement-records/sale-policy/	Farm/breed-level override
GET	/api/movement-records/sale-readiness/{animal_id}/	Full readiness evaluation, optional <code>expected_sale_price=</code>
GET	/api/movement-records/animal-profitability/{animal_id}/	Break-even, margin, return on cost
POST	/api/movement-records/sale-approval/{animal_id}/	Record required sale approval, with reason
POST	/api/movement-records/sale/ extended	Now supports a permission-gated <code>override_reason</code>

GET /api/movement-records/sale-readiness/{animal_id}/

RESPONSE • DATA

```
{
  "status": "sale_recommended",
  "restrictions": [],
  "manual_review_reasons": [],
  "factors": {
    "current_weight_kg": 32, "target_sale_weight_kg": 30,
    "age_months": 12.1, "growth_rate_kg_per_day": 0.09,
    "total_cost_to_date": 50000, "income_generated": 0,
    "withdrawal_end_date": null, "is_pregnant": false
  }
}
```

Status ladder — a restriction always wins over a manual-review flag, which always wins over the computed tier:

- `sale_restricted` — inactive/quarantined/serious health alert/drug withdrawal/disallowed pregnancy
- `manual_review_required` — pending treatment follow-up, disclosed pregnancy, or approval not yet obtained
- `not_ready_for_sale` → `approaching_sale_readiness` (≥85% of target weight) → `ready_for_sale` → `sale_recommended` (only once profit margin clears the configured threshold, when a price is given)

GET `/api/movement-records/animal-profitability/{animal_id}/`

Query params: `expected_sale_price`, `price_per_kg` (for break-even weight) — both optional; the figures that depend on them come back `null` without them, rather than a guess.

```
{
  "total_cost_to_date": 50000, "income_generated": 0,
  "break_even_price": 50000, "break_even_weight_kg": null,
  "expected_sale_expenses": 5000,
  "estimated_sale_profit": 45000,
  "estimated_profit_margin_pct": 45.0,
  "estimated_return_on_cost_pct": 90.0
}
```

10 • Reports

Fourteen read-only views over the ledger and engines above — nothing here computes anything new.

PATH	RETURNS
/api/reports/animal-cost/{page}/{size}/{farm_id}	Cost, income, profit/loss — one row per animal
/api/reports/cost-by-group/{farm_id}/	Cost totals per herd/group
/api/reports/feed-cost/{farm_id}/	Feed spend, total + by animal, optional date range
/api/reports/treatment-cost/{farm_id}/	Treatment spend, total + by animal
/api/reports/growth-performance/{page}/{size}/{farm_id}	ADG, % change, cost/kg — per animal
/api/reports/weight-exceptions/{farm_id}/	Animals with an open weight-related alert
/api/reports/reproductive-eligibility/{farm_id}/	Every active female, eligible or not, and why
/api/reports/sale-ready-animals/{farm_id}/	Status in {ready_for_sale, sale_recommended}
/api/reports/sale-restricted-animals/{farm_id}/	Status = sale_restricted, with reasons
/api/reports/profitability/{page}/{size}/{farm_id}	Break-even & cost figures per animal
/api/reports/low-performance-animals/{farm_id}/	Open poor-conversion / decline / reproductive-concern alerts
/api/reports/health-alert-summary/{farm_id}/	Open-alert counts by severity and type
/api/reports/expiring-drugs/{farm_id}/	Batches expiring within ? days_ahead= (default 30)
/api/reports/stock-valuation/{farm_id}/	Current feed + drug stock value, by batch

All 14 are **GET** and farm-scoped only — no separate permission gate beyond org membership, since none of them mutate anything.

11 · Audit & overrides

Every override capability follows the same shape: pass an `override_reason` , hold a permission distinct from (and in addition to) the base create right, and it's recorded — who, when, before, after, why.

WHERE OVERRIDES LIVE

ACTION	WHERE	PERMISSION (IN ADDITION TO BASE CREATE)
Override reproduction restriction	<code>POST /insemination/</code> , <code>POST /pregnancy/</code> — pass <code>override_reason</code>	<code>reproduction_restriction_override</code>
Override sale restriction	<code>POST /sale/</code> — pass <code>override_reason</code>	<code>sale_restriction_override</code>
Approve a sale	<code>POST /sale-approval/{id}/?reason=</code>	<code>update_sales_record</code>
Override lifecycle stage	<code>POST /lifecycle/animal/{id}/override/</code>	<code>update_animal_details</code>
Configure species rules	<code>POST /breeding-rule/</code> , <code>POST /sale-policy/</code>	<code>update_reproduction /</code> <code>update_sales_record</code>
Resolve a health alert	<code>POST /health-alert/{id}/resolve/</code>	<code>update_health</code>
Record external medication	<code>POST /treatment/external/</code>	<code>external_medication_override</code>
Edit a purchase value after the fact	<code>POST /animal/{id}/acquisition/</code> , called again	<code>update_animal_details</code>

Why two permissions, not one — the ability to override a restriction is deliberately layered on top of the ability to create the record at all, not a substitute for it. A user with only `reproduction_restriction_override` and no base create right is still blocked — matching how a vet's elevated authority in real life extends their normal duties rather than replacing them.

Restriction-based blocks stay hard blocks where overriding would corrupt state regardless of authorization — a sold or deceased animal can't be re-sold by any override; only the policy-level checks (withdrawal period, breeding age, pregnancy-and-sale policy) are override-eligible.